

2023 - 2024

Software Engineer Intern Portfolio

Ha-Dan (Na) Huynh

TABLE OF CONTENTS

01

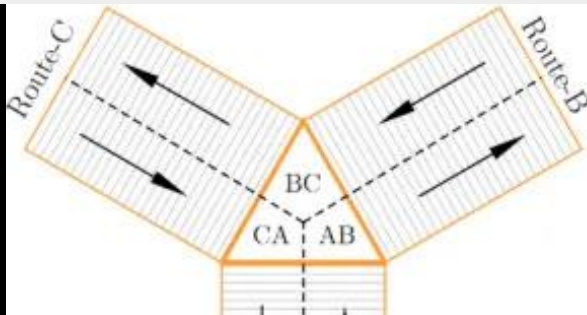


GameLoom CSV



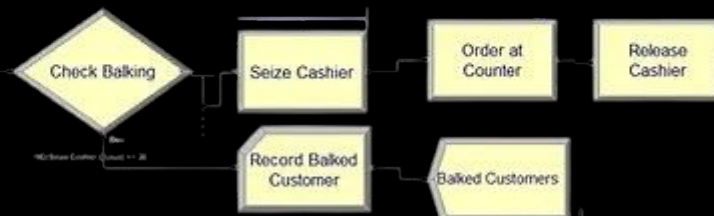
[GameLoom](#)

02



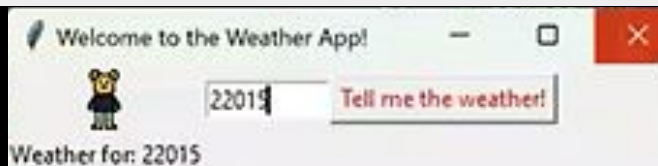
[Traffic Management](#)

03



[TeaDo Store](#)

04



[Weather API](#)

GameLoom

Date: Aug. 2024 – Present

Project Type: Course Project

Course: CS321: Software Engineering

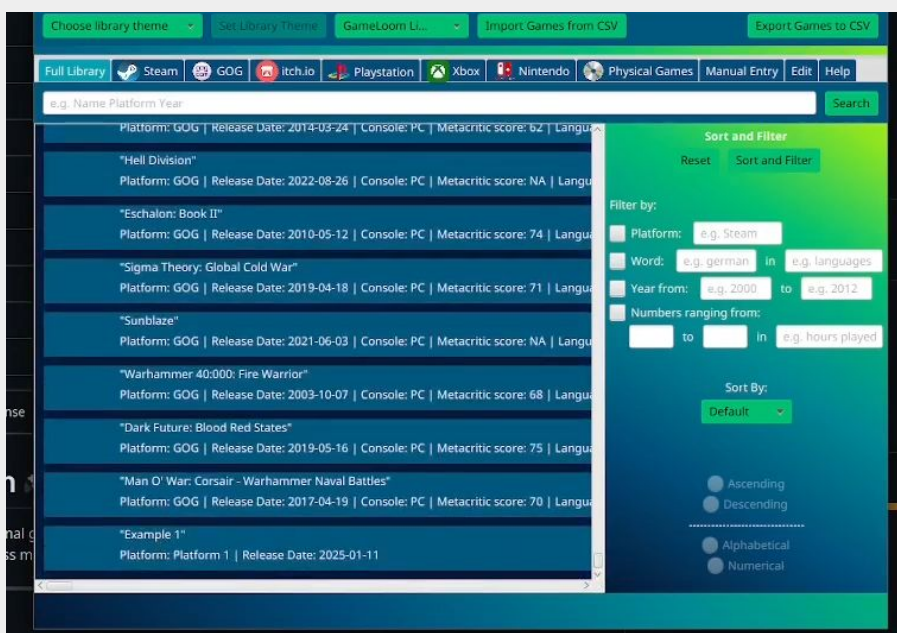
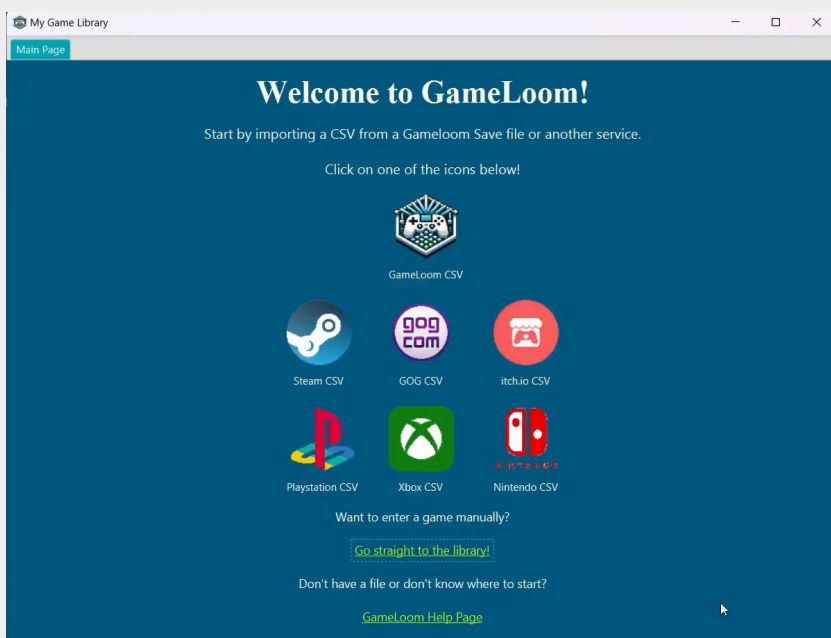
Languages/Technologies: Java, JavaFX, CSS

Technical Skills: Desktop application development, SQL & relational theory, UI design, Data structures & algorithms, Automation, Software Testing

Soft Skills: Agile methodology, Software Development Life Cycle, Problem-Solving, Communication

Description: GameLoom is a personal game library manager that acts as a unified catalog for tracking physical and digital game collections across multiple platforms.

[Github Repository](#)



Traffic Management Simulation

Date: Spring 2024

Project Type: Course Project

Course: CS 571 Operating Systems

Language: C

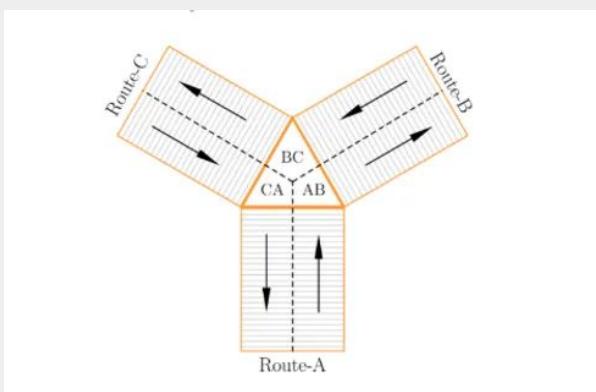
Technical Skills: Concurrency, Multithreading, Software Design, Testing

Soft Skills: Problem Solving, Analytical Thinking, Commitment to Excellence

Description:

- Implemented a stoplight program that manages a three-way intersection exemplifying software design and synchronization primitives.
- Developed priority-base functionality to ensure high-priority vehicles approaching the intersection through the same route cross the intersection first.
- Optimized throughput but enabling thread concurrency, allowing multiple vehicles cross the intersection while preventing collisions.
- Exceed performance expectations and successfully passed 285 test cases with metrics of:
 - Crash Rate: 0.00%
 - Priority Violation Rate: 0.32%
 - Missing Vehicle Rate: 0.00%
 - Premature Termination Rate: 0.00%

Supplementary Images



```
start(100)
Car    2 from C turning left : approach()
Truck  3 from B turning right: approach()
Car    0 from A turning right: approach()
Car    4 from B turning left : approach()
Truck  12 from A turning left : approach()
Car    14 from B turning left : approach()
Truck  5 from C turning right: approach()
Car    51 from C turning right: approach()
Car    1 from C turning right: approach()
Truck  9 from B turning right: approach()
Car    2 from C turning left : enter(CA)
Car    8 from A turning left : approach()
Truck  6 from C turning left : approach()
Truck  7 from A turning left : approach()
Car    16 from B turning right: approach()
Car    0 from A turning right: enter(AB)
Truck  13 from B turning left : approach()
```

TeaDo Store – Simulation of a Bubble Tea Restaurant

Date: Spring 2024

Project Type: Course Project

Course: SYST 335 – Discrete Systems Simulation Modeling

Language: Python

Skills: Back-End Coding, Simulation, Optimization

Description: This model is discrete event simulation for TeaDo Fairfax, a local Bubble Tea Restaurant. The project aimed to improved customer satisfaction by reducing customer waiting times through modeling software and Python programming. This involved analyzing walk-in orders and identifying delay factors in the service process. Utilizing resource allocation and simulation techniques, the project maximized the restaurant's work flow and was able to reduce customer balking (customers leaving due to long wait times) by 20%. Python was used to implement aspects of a M/M/1 Queueing System such as Poisson arrivals, servers with different service time distributions, and a FIFO queue.

Basic Queueing Statistics & Theory

Queue with exponential (Poisson) server

```
Clock: 1000001.9158071107
Server utilization is: 0.5010112254265631
Average waiting time in queue is: 4.959485369068653
Average number of customers in queue: 0.4962153984638062
Maximum number of customers in queue is: 15
```

Service Time Distribution with Uniform Distribution

```
Server utilization is: 0.4146614336104698
Average waiting time in queue is: 1.56014727396945
Average number of customers in queue: 0.15596318628331776
Maximum number of customers in queue is: 8
```

Number of customers in the queue excluding the customer in service

```
95% T-Confidence Interval for average number of customers in queue:
Mean: 1.05, Lower bound of CI: 1.03, Upper bound of CI: 1.07
95% T-Confidence Interval for average waiting time in queue:
Mean: 10.52, Lower bound of CI: 10.34, Upper bound of CI: 10.70
```

```
[Done] exited with code=0 in 9.545 seconds
```


Code Sample: Aspects of a FIFO Queue

```
class Queue(Event): #FIFO Queue for customer arrivals
    def __init__(self):
        self.queue = []

    def isEmpty(self):
        return len(self.queue) == 0

    def enqueue(self, evnt):
        self.queue.append(evnt)

    def dequeue(self):
        return self.queue.pop(0)
```

```
class Simulator:
    def __init__(self):
        self.clock = 0
        self.number_in_service = 0
        self.queue_length = 0
        self.last_event_time = 0.0 #time elapsed from last event to current event
        self.total_busy = 0.0 #total amt of time server has been busy
        self.max_queue_length = 0
        self.sum_response_time = 0.0 #how many customers have left
        self.number_of_departure = 0.0
        self.sum_wait_time = 0.0 #total waiting time of customers
        self.weighted_queue_length = 0.0 #calculates average queue length

        self.number_of_customers = 0 #total customers (may be past simulated time period)
```

```
def process_arrival(self, evt):
    self.customers.put(evt)

    # Updating weighted queue length and total busy time
    self.weighted_queue_length += (self.clock - self.last_event_time) * self.queue_length
    self.total_busy += (self.clock - self.last_event_time) * self.number_in_service

    self.queue_length += 1

    if self.number_in_service < 2:
        self.schedule_departure()

    # Updating maximum queue length
    if self.max_queue_length < self.queue_length:
        self.max_queue_length = self.queue_length
```

```
# Running multiple simulations and report the results
def run_simulations(num_simulations, simulation_time):
    results = [] # Initialize an empty list to store the results of each simulation.
    for _ in range(num_simulations): # Loop over the specified number of simulations.
        sim = Simulator() # Create a new instance of the Simulator class.
        result = sim.run_simulation(simulation_time) # Run the simulation for the specified duration and store the result.
        results.append(result) # Add it to the list of results.
    return pd.DataFrame(results) # Convert the list of results into a Pandas DataFrame and return it.
```

Weather Recommendations

Date: October 2023

Project Type: Hackathon Submission

Hackathon: PatriotHacks 2023-2024 at George Mason University

Language: Python

Skills: Front-End Coding, GUI, API Requests

Description: This Python program was created for Patriothacks 2023's Hackathon. This program is designed to provide personalized clothing recommendations based on the national weather forecast for a user-provided location. It utilizes API calls to fetch 8 hours of weather forecast data to suggest appropriate clothing options for the user. Worked closely with other members with the UI design team to ensure an intuitive user experience.

[Github Repository](#)



```
class weather_API:
    def __init__(self):
        # gets the averages of the parameters from an 8 hour period given a zipcode
    def get_averages (self, zipcode):
        url = f"https://api.tomorrow.io/v4/weather/forecast?location={zipcode}&US&apikey=o2Vfm23"
        headers = {"accept": "application/json"}
        for i in range(8) :
            response = requests.get(url, headers=headers)
            data = response.json()
            self.avg_temperature += (data['timelines']['hourly'][i]['values']['temperature'])
            self.avg_temperatureApparent += (data['timelines']['hourly'][i]['values']['temperatureApparent'])
            self.avg_humidity += (data['timelines']['hourly'][i]['values']['humidity'])
            self.avg_rainAccumulation += (data['timelines']['hourly'][i]['values']['rainAccumulation'])
            self.avg_snowAccumulation += (data['timelines']['hourly'][i]['values']['snowAccumulation'])
            self.avg_uvIndex += (data['timelines']['hourly'][i]['values']['uvIndex'])
            self.avg_windGust += (data['timelines']['hourly'][i]['values']['windGust'])
            self.avg_windSpeed += (data['timelines']['hourly'][i]['values']['windSpeed'])
            if (data['timelines']['hourly'][i]['values']['windSpeed']) == 1:
                self.if_uvHealthConcern = 1
        self.avg_temperature /= 8
        self.avg_temperatureApparent /= 8
        self.avg_humidity /= 8
        self.avg_rainAccumulation /= 8
        self.avg_snowAccumulation /= 8
        self.avg_uvIndex /= 8
        self.avg_windGust /= 8
        self.avg_windSpeed /= 8
        self.data_list[0] = self.avg_temperature
        self.data_list[1] = self.avg_temperatureApparent
        self.data_list[2] = self.avg_humidity
        self.data_list[3] = self.avg_rainAccumulation
```



Ha-Dan (Na) Huynh

In this software engineering portfolio, I invite you to explore a diverse collection of projects that reflect my passion for design, creativity, and innovation.

CONTACT ME

+1 703 323 8107

<https://mason.gmu.edu/~hhuynh22/>

hhuynh22@gmu.edu

[Ha-Dan \(Na\) Huynh | LinkedIn](#)

9773 Lakepointe Dr Burke VA 22015

